

Short Term Residential Load Forecasting Based on LSTM Recurrent Neural Network

Gaurav Yadav, 21075034 K.V.K Varun, 21075044

Under the guidance of

Jaishankara sir



Department of Computer Science & Engineering

INDIAN INSTITUTE OF TECHNOLOGY (BHU) VARANASI

Varanasi 221005, India

3rd May 2023

Acknowledgments

We would like to express my heartfelt gratitude to all those who have supported and helped us during the preparation of this report.

First and foremost, we would like to thank our supervisor **Jaishankar Sir**, for providing us with invaluable guidance and support throughout the entire project. Their encouragement and feedback have been instrumental in shaping the direction and focus of this report.

We are also grateful to our fellow students and friends who have provided us with constant encouragement and support during the entire project. Their feedback and suggestions have been of immense help in improving the quality and clarity of this report.

Without the support and encouragement of these individuals and organizations, this report would not have been possible. Once again, we express our sincere gratitude to all those who have contributed to this report.

- GAURAV YADAV

K.V.K VARUN

Abstract

The paper addresses an important issue in the field of energy management, namely the short-term residential load forecasting. The ability to accurately forecast electricity demand is essential for the efficient operation of the power grid and the optimization of energy resources. The authors propose an innovative approach that uses the LSTM network, a powerful deep learning technique that has been shown to be effective in time-series forecasting. The method considers multiple factors that can influence the electricity consumption of households, such as weather conditions, time of day, and historical consumption patterns. The proposed method is tested on real-world data from residential households, demonstrating its practicality and effectiveness. The results of the study show that the LSTM-based method outperforms traditional forecasting models, highlighting the potential of deep learning techniques for energy forecasting applications. The implications of this work are significant, as it can contribute to the development of more efficient and sustainable energy management systems. The paper concludes by discussing potential future research directions and the practical implementation of the proposed method in real-world scenarios.

INTRODUCTION

The current power system is shifting towards greater sustainability, but this is introducing additional complexity and uncertainty due to factors such as the growing use of renewables, electric vehicles, and fluctuating load demands in distribution grids. In order to achieve this goal, precise forecasting of short-term electric loads at the residential level can greatly assist in the management of power system operations. By accurately forecasting loads, it becomes possible to utilize energy storage (ES) systems or intelligent demand response (DR) technologies in a flexible manner, which enables peak load shaving. If accurate load forecasting is achieved, utilities can have greater confidence in incentive-based demand response (DR) programs that reduce demand, which is important for providing load-balancing reserves and/or hedging market costs. Historically, forecasting loads at the customer level was not viewed as important due to the unpredictable nature of individual loads. As a result, there are still unresolved issues related to forecasting short-term household loads.

In recent times, deep learning has emerged as a highly active area of research across multiple fields. Unlike shallow learning, deep learning involves stacking multiple layers of neural networks and utilizing stochastic optimization to perform machine learning tasks. A greater number of layers can provide different levels of abstraction, leading to improved learning ability and task performance [1]. Among deep learning methods, the long short-term memory (LSTM) recurrent neural network (RNN) has garnered significant attention in the area of sequence learning. Originally introduced by Hochreiter and Schmidhuber [2], effective applications based on LSTM networks have been demonstrated in diverse areas, including natural language translation [3], image captioning, and speech recognition [4]. Although most of these applications are focused on data classification, there have been some limited applications of LSTM networks on regression tasks.

The paper aims to address the issues of short-term residential load forecasting by conducting an exploratory analysis of customer-wise data and evaluating the inconsistency in residential load profiles. A density-based clustering technique is employed to indicate the difficulty of load forecasting. The paper then proposes a residential load forecasting framework based on LSTM and develops a conservative empirical optimization-based predictor. The study demonstrates that LSTM outperforms other machine learning-based predictors and produces the best forecasts for the majority of cases. Aggregating individual forecasts is generally found to be more accurate than directly forecasting the aggregated loads through the proposed approach.

LITERATURE STUDY

In the field of short-term load forecasting, numerous studies have been conducted. For instance, Cao et al [5]. utilized the autoregressive integrated moving average (ARIMA) model and a similar day technique for intraday load prediction. Their similar day method involves grouping the day being forecasted with other days in the past that have similar meteorological conditions, and then predicting the load based on the average demand of those days. They showed that ARIMA is more accurate in typical days, while the similar day method is more effective in unusual days. Radial basis function (RBF) neural network had been used to address the short-term load forecasting in [6] and [7]. Yun *et al.* combine the RBF neural network with the adaptive neural fuzzy inference system (ANFIS) to adjust the prediction by taking account of the real-time electricity price.

Li *et al.* [7] addressed the short-term day-ahead forecasting problem via a grid method combined with the back propagation (BP) and the RBF neural networks. The grid method is similar to the similar day method in [5], but instead of grouping days with similar meteorological measurement, it groups load profiles according to location, nature and size of the loads. For each group, they trained a BP network and an RBF network to predict the day-ahead load demand.

Recently, k -nearest neighbour (KNN) algorithm had also seen some successful examples on load forecasting [8], [9], whose dominant advantage is its efficiency. Ghofrani *et al.* [10] proposed a dedicated input selection scheme to work with the hybrid forecasting framework using wavelet transformation and Bayesian neural network. To the best of our knowledge, this approach achieved the most accurate forecasting performance on system-level load forecasting, with mean absolute percentage error (MAPE) score as small as 0.419% on average.

Numerous studies on short-term load forecasting have been conducted, but only a few have focused on individual user load forecasting. Chaouch [11] and Ghofrani et al. [12] were the first to examine individual user load forecasting, with Chaouch proposing a functional time series forecasting approach and Ghofrani using a Kalman filter estimator. While Chaouch reported daily median absolute errors, they did not use the more common mean absolute percentage error (MAPE) metric, making it unsuitable as a benchmark for comparing results. Ghofrani's work achieved an MAPE ranging from 18% to 30%. Recently, deep learning methods have been used to improve load forecasting accuracy, with Ryu et al. [13] achieving an 8.85% MAPE for industrial customers. Mocanu et al. [14] and Marino et al. [15] used factored conditional restricted Boltzmann machine and LSTM, respectively, for single-meter residential load forecasting and demonstrated notable improvements in performance compared to shallow neural networks and support vector machines. However, their effectiveness was only verified on the metric of root mean square error (RMSE), and they lacked justifiable discussion of their choice of method. Additionally, all these studies were only verified with a single residential household, making it impossible to evaluate the aggregated effect of customer-wise load forecasts.

OBJECTIVE

The objective is to propose a deep learning-based approach, specifically using Long Short-Term Memory (LSTM) recurrent neural network, for short-term load forecasting of residential customers. The paper aims to evaluate the performance of the proposed approach and compare it with other existing methods using metrics such as Mean Absolute Percentage Error (MAPE) and Root Mean Square Error (RMSE). The paper presents a load forecasting framework for the difficult task of predicting individual residential load, using LSTM recurrent neural network, since LSTM has demonstrated its ability to understand long-term temporal relationships.

PROPOSED APPROACH

A. LSTM Based Model

To briefly introduce the concept of the LSTM, we adopt a similar naming convention as in [16]. Let $\{x_1, x_2, \dots, x_T\}$ denote a typical input sequence for an LSTM, where $x_t \in \mathcal{R}^\kappa$ represent a κ -dimensional vector of real values at the t th time step. In order to establish temporal connections, the LSTM defines and maintains an internal memory cell state throughout the whole life cycle, which is the most important element of the LSTM structure. The memory cell state s_{t-1} interacts with the intermediate output h_{t-1} and the subsequent input x_t to determine which elements of the internal state vector should be updated, maintained or erased based on the outputs of the previous time step and the inputs of the present time step. In addition to the internal state, the LSTM structure also defines input node g_t , input gate i_t , forget gate f_t and output gate o_t . The formulations of all nodes in an LSTM structure are given by (1) to (6):

$$\begin{aligned} f_t &= \sigma(W_{fx}x_t + W_{fh}h_{t-1} + b_f) \\ i_t &= \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i) \\ g_t &= \phi(W_{gx}x_t + W_{gh}h_{t-1} + b_g) \\ o_t &= \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o) \\ s_t &= g_t \odot i_t + s_{t-1} \odot f_t \\ h_t &= \phi(s_t) \odot o_t \end{aligned}$$

Where W_{gx} , W_{gh} , W_{ix} , W_{ih} , W_{fx} , W_{fh} , W_{ox} and W_{oh} are weight matrices for the corresponding inputs of the network activation functions; $\odot$ stands for an element-wise multiplication; σ represents the sigmoid activation function, while ϕ represents the tanh function.

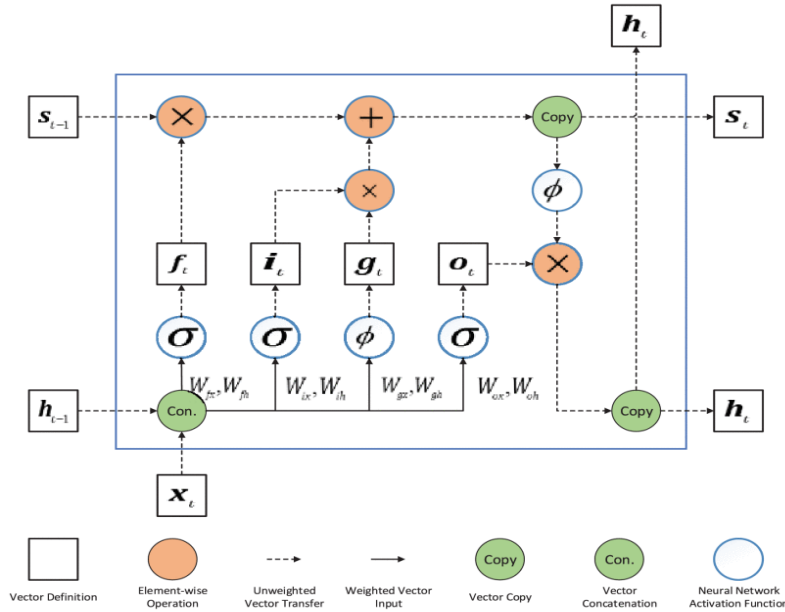


Figure 1 Structure of a LSTM block

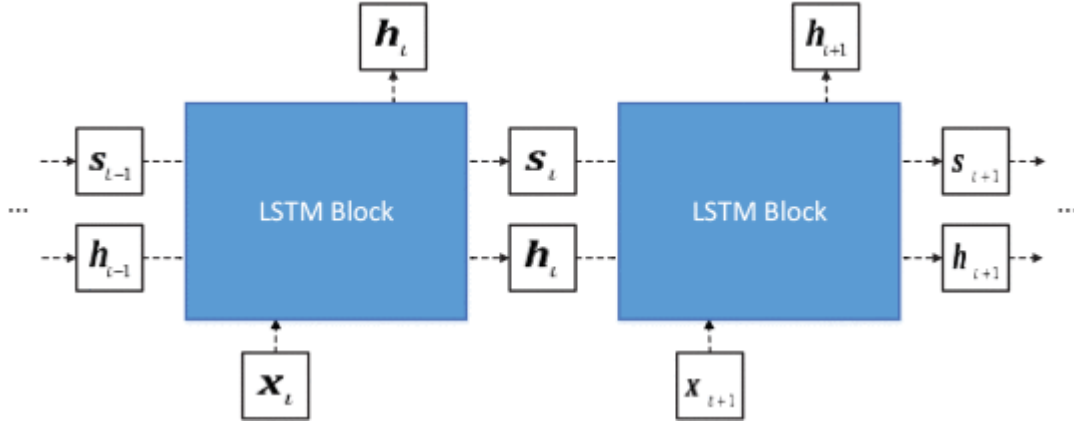


Figure 2 unrolled LSTM sequential structure

To train a simple one-layer LSTM recurrent network, one should specify the hyperparameter of the hidden output dimension n . In this case, the hidden output at a given time step $\mathbf{h}_t \in \mathcal{R}^n$ which is a n -dimensional vector, so as to $\mathbf{s}_t$. The common initialization for $\mathbf{h}_t$ and $\mathbf{s}_t$ is zero initialisation. If the gate is 0, then the signal is blocked by the gate. The decisions for the forget gate $\mathbf{f}$, the input gate $\mathbf{i}$ and the output gate $\mathbf{o}$ are all dependent on the current input $\mathbf{x}_t$ and the previous output $\mathbf{h}_{t-1}$. The signal of the input gate controls what to preserve in the internal state, while the forget gate controls what to forget from the previous state $\mathbf{s}_{t-1}$. With the internal state updated, the output gate decides which the internal state $\mathbf{s}_t$ should pass as the LSTM output $\mathbf{h}_t$. This process then continues to repeat for the next time step.

B. The LSTM Based Forecasting Framework

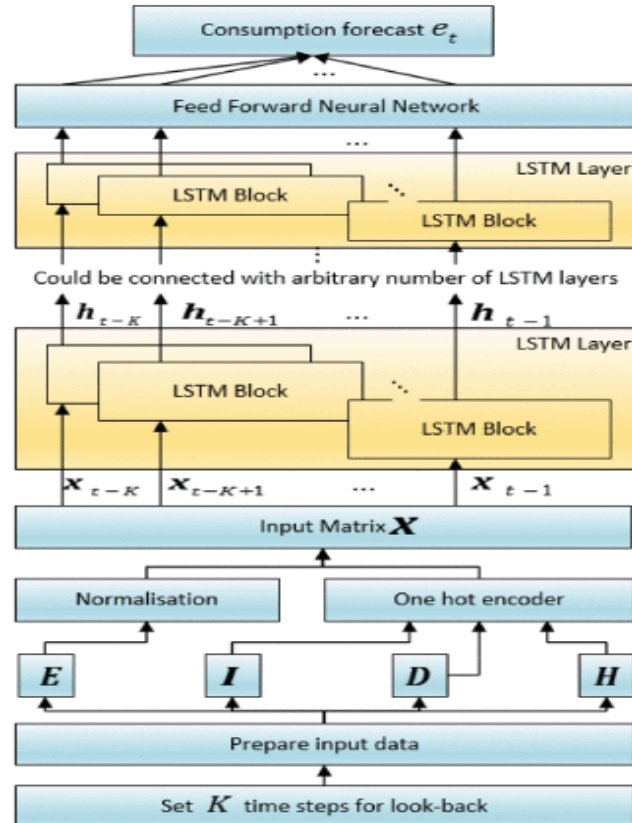


Figure 3 Our LSTM forecasting framework

The framework starts from data pre-processing for the inputs. In this study, the following input features are used:

1. the sequence of energy consumptions for the past k time steps

$$\mathbf{E} = \{e_{t-K}, \dots, e_{t-2}, e_{t-1}\} \in \mathbb{R}^K$$

2. the incremental sequence of the time day indices for the past k time steps $\mathbf{I} \in \mathbb{R}^K$, where the range for each element of $\mathbf{I}$ is (1 to 48);
3. the corresponding day of week indices for the past k time steps $\mathbf{D}$, each of which ranges from 0 to 6;
4. the corresponding binary holiday marks $\mathbf{H}$, each of which can either be 0 or 1.

Once the four vectors are scaled to $\tilde{\mathbf{E}}, \tilde{\mathbf{I}}, \tilde{\mathbf{D}}, \tilde{\mathbf{H}}$, the input for LSTM layer is a matrix of the concatenation of the four, i.e.:

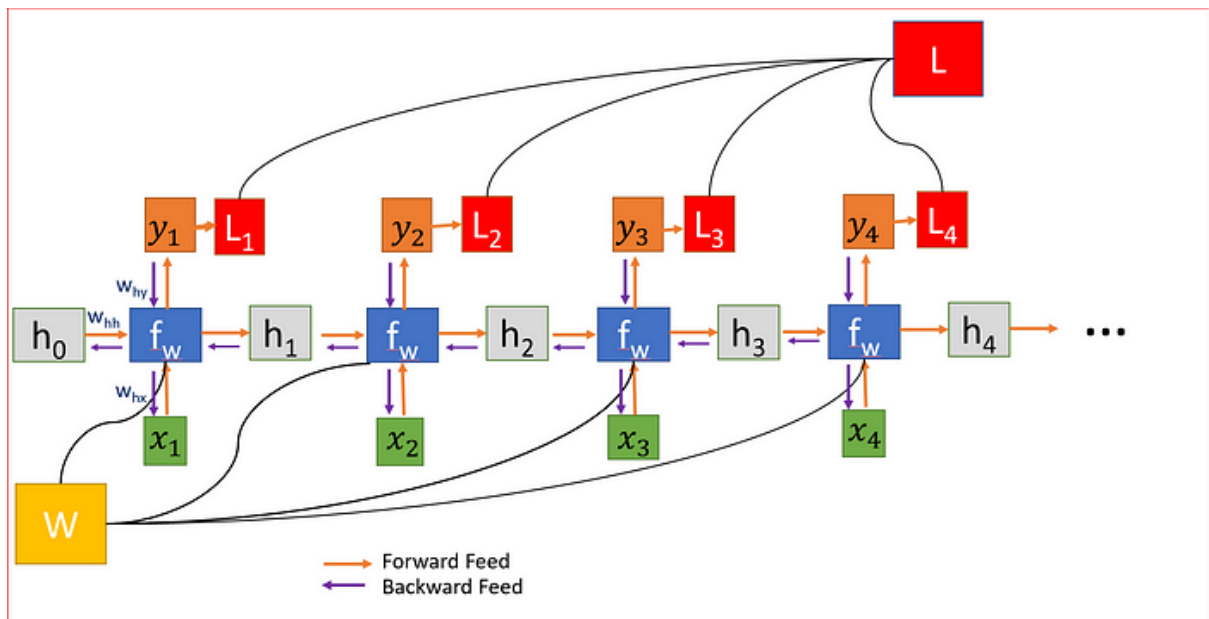
$$\mathbf{X} = \{\tilde{\mathbf{E}}^T, \tilde{\mathbf{I}}^T, \tilde{\mathbf{D}}^T, \tilde{\mathbf{H}}^T\}$$

Each row of the input matrix is the scaled features for the corresponding time step, which is fed to corresponding LSTM block in the LSTM layer. Due to the sequential nature of the output of an LSTM layer, an arbitrary number of LSTM layers can be stacked to form a deep learning network.

The outputs of the top LSTM layer are fed to a conventional feedforward neural network which maps the intermediate LSTM outputs to a single value, i.e., the energy consumption forecast of the target time interval.

C. The GRU Model

By using deeper neural layers in RNN and weight sharing across different RNN cells, we can accumulate the gradients at each time step. As gradients are processed through matrix multiplication due to the chain rule, they may either decrease exponentially and become very small, which is known as Vanishing gradient, or increase to very large values, which is referred to as Exploding gradients.



Vanishing Gradients is addressed either Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU). Both GRU and LSTM have gating mechanisms that enable them to control the flow of information and remember contextual information over multiple time steps. These mechanisms allow them to determine which pieces of information from the past should be retained and which should be discarded. GRU utilizes Update and Reset gates to accomplish this.

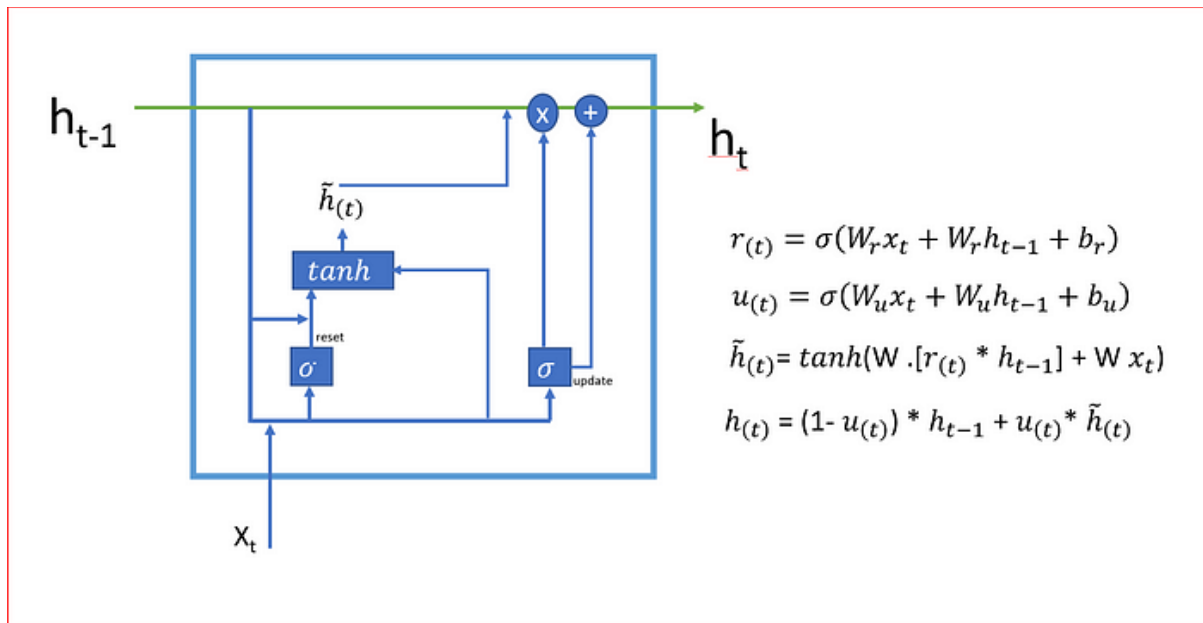


Figure 4 The GRU architecture

Reset Gate takes the input x_t and the previously hidden state h_{t-1} and applies a sigmoid activation function. Reset gate determines if the current state will have the new information or will still have the previous information

Update Gate takes the input x_t and the previously hidden state h_{t-1} and applies a sigmoid activation function.

Memory cell in RNN, GRU and LSTM	
$h_t = f_\omega(h_{t-1}, x_t)$	RNN
$h_t = (1 - u(t)) * h_{t-1} + u(t) * \tilde{h}(t)$ Where $\tilde{h}(t)$	GRU
$\tilde{h}(t) = \tanh(W \cdot [r(t) * h_{t-1}] + W x_t)$	
$C_t = f_t \odot C_{t-1} + i_t \odot g_t$ Where	LSTM
$f_t = \sigma(W_{fx} x_t + W_{fh} h_{t-1} + b_f)$	
$i_t = \sigma(W_{ix} x_t + W_{ih} h_{t-1} + b_i)$	
$g_t = \tanh(W_{gx} x_t + W_{gh} h_{t-1} + b_g)$	

D. Regression model

Regression models are employed to explain how variables relate to one another by matching a line to the data that has been observed. Regression enables one to approximate how a dependent variable varies as independent variable(s) alter.

Multiple Linear Regression: The formula for a multiple linear regression is: -

$$y = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n + \epsilon$$

- y = the predicted value of the dependent variable
- B_0 = the y-intercept (value of y when all other parameters are set to 0)
- $B_1 X_1$ = the regression coefficient (B_1) of the first independent variable (X_1) (a.k.a. the effect that increasing the value of the independent variable has on the predicted y value)
- ϵ = model error (a.k.a. how much variation there is in our estimate of y)

Polynomial Regression: In polynomial regression, the relationship between the independent variable x and the dependent variable y is described as an nth degree polynomial in x. Polynomial regression, abbreviated $E(y | x)$, describes the fitting of a nonlinear relationship between the value of x and the conditional mean of y. It usually corresponded to the least-squares method.

$$y = \theta_0 + \theta_1 x \rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$$

Because the weights associated with the features are still linear, this is still called a linear model. x^2 (x square) is only a function. However, the curve we're trying to fit is quadratic in nature.

$$\text{Cost function, } J = \frac{1}{n} \sum_{i=1}^n (\text{Predicted value} - \text{Expected value})^2$$

APPLYING THE MODEL AND RESULTS

DISCUSSION

The ‘[Household Power Consumption](#)’ dataset is a multivariate time series dataset that describes the electricity consumption for a single household over four years. The data was collected between December 2006 and November 2010 and observations of power consumption within the household were collected every minute.

It is a multivariate series comprised of seven variables (besides the date and time); they are:

- **global_active_power:** The total active power consumed by the household (kilowatts).
- **global_reactive_power:** The total reactive power consumed by the household (kilowatts)
- **voltage:** Average voltage(volts).
- **global_intensity:** Average current intensity (amps).
- **sub_meeting_1:** Active energy for kitchen (watt-hours of active energy).
- **sub_meeting_2:** Active energy for laundry (watt-hours of active energy).
- **sub_meeting_3:** Active energy for climate control systems (watt-hours of active energy).

EVALUATION METRIC

1. **Empirical means:** We begin with one of the most basic predictors, which is the empirical mean predictor. This predictor generates a forecast value by using the statistical mean for a particular customer ID, time of day, and day type (weekday or weekend).
2. **Empirical MAPE Minimisation:** Similar to the empirical mean predictor , the empirical MAPE minimisation predictor is based on statistical energy consumption distribution. With the empirical pmf ready, the MAPE expectation of any forecasts can be calculated. Let p denote an arbitrary predicted load value within the empirical range and p_i denote the i th possible discretised value also within the empirical range, the MAPE expectation of choosing arbitrary predicted load value p as the forecast is

$$\mathcal{E}_{id,t,h}(p) = \sum_i pmf_{id,t,h}(p_i) \left| \frac{(p - p_i)}{p_i} \right|$$

$$\hat{p}_t = \underset{p}{\operatorname{argmin}}(\mathcal{E}_{id,t,h}(p))$$

The LSTM Model:

- The first layer is an LSTM layer with 200 units and an input shape of (X_train.shape[1], X_train.shape[2]). The input shape is determined by the number of time steps (X_train.shape[1]) and the number of features (X_train.shape[2]) in the training data. This layer returns sequences of outputs.
- A dropout layer is added after the first LSTM layer to regularize the model and prevent overfitting. The dropout rate is set to 0.2, which means that 20% of the inputs to this layer will be randomly set to zero during training.
- The second layer is another LSTM layer with 200 units that does not return sequences of outputs.
- Another dropout layer is added after the second LSTM layer to further regularize the model. The dropout rate is set to 0.3.
- The final layer is a dense layer with a single output unit, which is the predicted value.

Layer (type)	Output Shape	Param #
=====		
=====		
lstm (LSTM)	(None, 1, 200)	165600
dropout (Dropout)	(None, 1, 200)	0
lstm_1 (LSTM)	(None, 200)	320800
dropout_1 (Dropout)	(None, 200)	0
dense (Dense)	(None, 1)	201
=====		
=====		
Total params: 486,601		
Trainable params: 486,601		
Non-trainable params: 0		

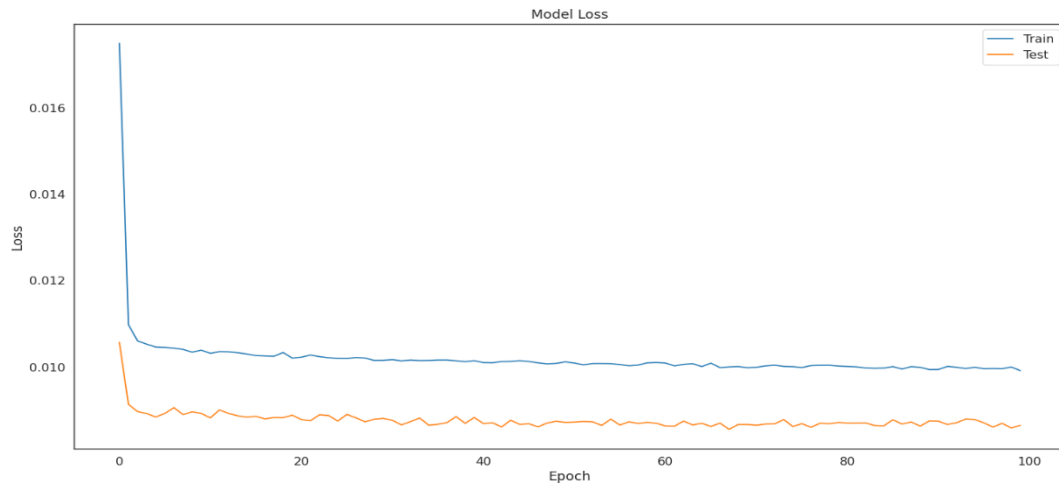


Figure 6 Model loss

Evaluation metrics	Value
RMSE	0.598
MAE	0.428
R^2	0.506

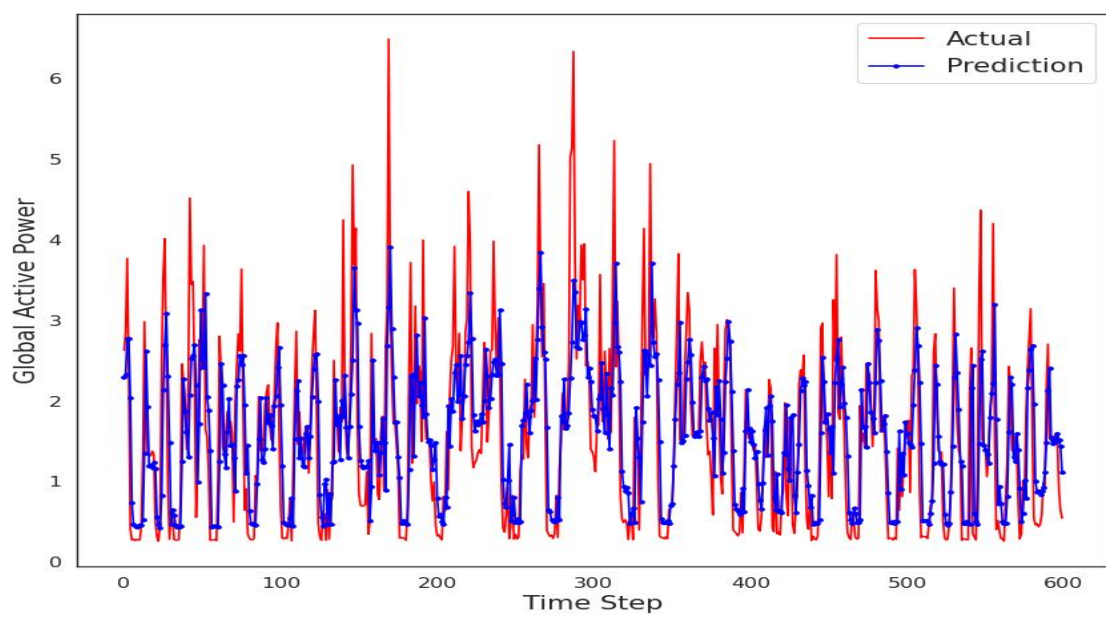


Figure 5 Actual vs the forecast values

The GRU Model:

- The Sequential function initializes a linear stack of layers. The first layer added to the model is a GRU layer with 75 units and return sequence is selected True, which means it returns the full sequence of outputs for each input time step. The input shape parameter defines the shape of the input data as a 3D tensor.
- The next two layers are also GRU layers with 30 units each, and return sequences is True. The last layer is a Dense layer with one unit, which outputs the predicted value for the next time step.
- The model is compiled using the mean absolute error (loss='mae') as the loss function and the Adam optimizer. The mean absolute error measures the absolute difference between the predicted and actual values, and is commonly used in regression problems.

The GRU model summary :

Layer (type)	Output Shape	Param #
=====		
=====		
gru_3 (GRU)	(None, 1, 75)	18900
gru_4 (GRU)	(None, 1, 30)	9630
gru_5 (GRU)	(None, 30)	5580
dense_3 (Dense)	(None, 1)	31
=====		
=====		
Total params: 34,141		
Trainable params: 34,141		
Non-trainable params: 0		

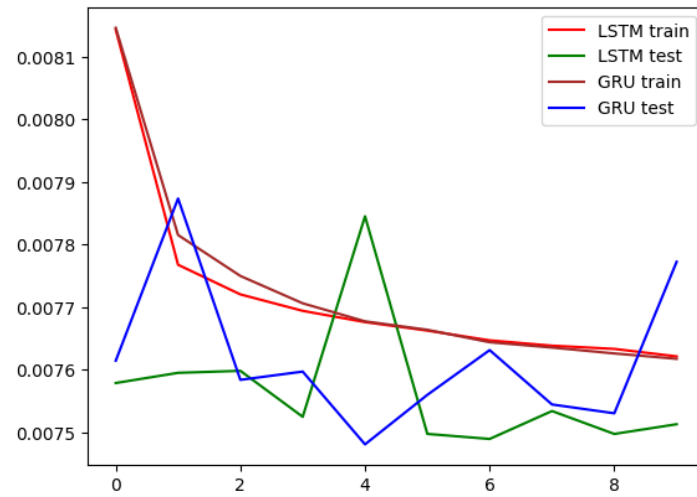


Figure 7 comparison of the GRU and LSTM models

Evaluation Metrics	Values
RMSE	0.263
MAE	0.086
R^2	0.938

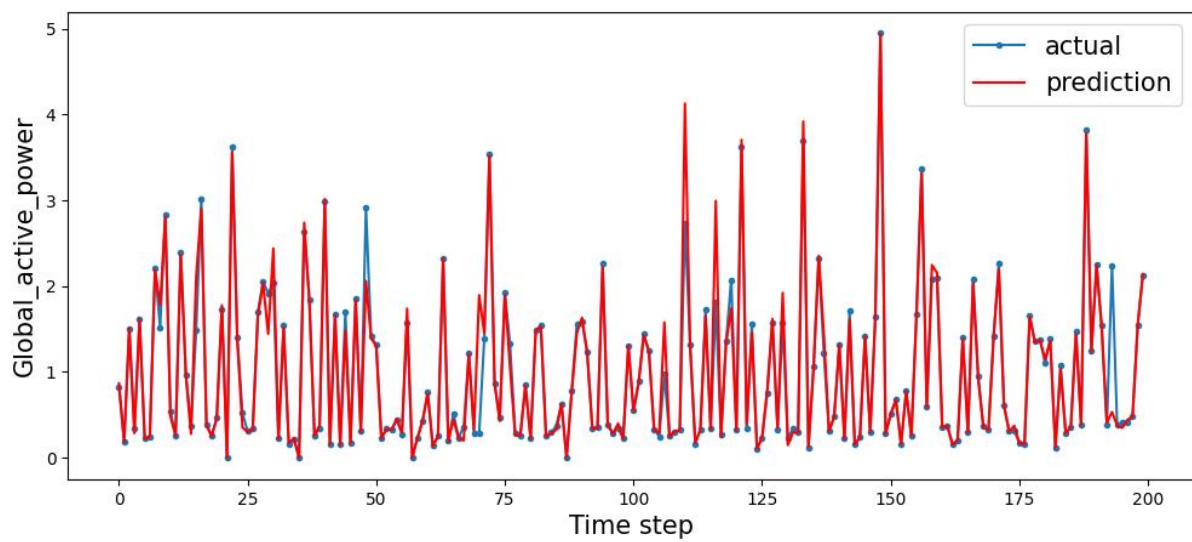


Figure 8 Actual vs the forecast value for GRU model

The Regression Model:

a. Multiple Linear Regression:

- 'linmodel' class is defined with two parameters: df and target.
- 'pre_processing' method is defined to preprocess data by splitting it into training and testing sets.
- The X_train, X_test, Y_train, and Y_test variables are set using the train_test_split function.
- 'fit_pred_acc' method is defined to fit the linear regression model, predict the target variable, and calculate evaluation metrics.
- 'LinearRegression()' is used to create a linear regression object.
- 'reg.fit' method is used to fit the linear regression model using the training data.
- 'reg.predict' method is used to predict the target variable using the testing data.
- 'skm.mean_absolute_error, skm.mean_squared_error, skm.r2_score, and skm.explained_variance_score' functions are used to calculate Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R-squared, and explained variance (EV) respectively.
- The evaluation metrics are returned in a list.
- An instance of the 'linmodel' class is created and its methods are called to preprocess data and fit the linear regression model.
- The evaluation metrics are stored in a dictionary with the key "Mult. Reg".

b. Polynomial Linear Regression:

It is also similarly modelled as the multiple linear regression except for the functions we used are for the polynomial linear regression.

c. Lasso And Ridge Regression:

similar methods are used for these models also except for some parameters change in these models which are required for particular model.

	MAE score	RMSE score	R ² score	Expl. Var
Multiple Regression	0.03	0.04	0.9985	0.9985
Ridge Regression	0.03	0.04	0.9985	0.9985
Lasso Regression	0.17	0.23	0.9509	0.9509
Polynomial Regression	0.02	0.04	0.9988	0.9988

CONCLUSION

This paper addresses the problem of forecasting short-term load for individual households. The article highlights the differences between electric load at a granular level and load at a substation level. Daily consumption patterns exist at the system or substation level, but energy consumption in a single household is often unpredictable. The article uses a density-based clustering technique to measure and compare the inconsistency between the aggregated load and individual loads. Based on the Practice Theory, which suggests that repetitive patterns reflect residents' lifestyles in energy consumption, the paper proposes an LSTM recurrent neural network-based framework for the challenging task of individual residential load forecasting. LSTM is chosen for this task because it can learn long-term temporal connections.

Multiple benchmarks are comprehensively tested and compared to the proposed LSTM load forecasting framework on a real-world dataset. We also tested different other models like the GRU model and the regression models. The GRU model is like the LSTM model with less complexity and only two gates are required to handle the information passing into the cell states.

We found that the GRU model is performing better than the other method after comparing the different evaluation metrics we applied and found. The inconsistency and unpredictability of the everyday usage of the electricity by individuals greatly effects the forecasting accuracy. So the higher the inconsistency the higher we need the demand of LSTM to perform the forecasting in place of the old backpropagation techniques.

For the future works we can tune the parameters more and this can further develop more accuracy in the forecasting of the values for different types of customers who have different style of livelihood and different patterns of living than the general